

Làm cho hiệu suất thuật toán “nhảy lên”: thao tác và ứng dụng của skip list

Wei Ran

Trường Trung học số 2 trực thuộc Đại học Sư phạm Hoa Đông, Thượng Hải

2005

Nguồn gốc: Làm cho hiệu suất thuật toán “nhảy lên”: thao tác và ứng dụng của skip list

IOI China candidate team papers / 2005 / Wei Ran.doc

Tóm tắt

Bài viết gồm ba phần chính. Phần đầu giới thiệu skip list từ chức năng, hiệu suất và cấu trúc trực quan, để người đọc có hình dung ban đầu về cấu trúc dữ liệu này. Phần thứ hai trình bày ba thao tác cơ bản: tìm kiếm, chèn và xóa, đồng thời phân tích độ phức tạp thời gian và bộ nhớ của chúng. Phần thứ ba giới thiệu ứng dụng của skip list, dùng kết quả thử nghiệm để so sánh skip list với một số cấu trúc dữ liệu liên quan, từ đó nêu ưu nhược điểm của từng cách tiếp cận. Cuối cùng, bài viết rút ra vài nhận xét về giá trị thực hành của skip list.

Từ khóa: skip list; hiệu suất; xác suất; ngẫu nhiên hóa.

1 Tổng quan và cấu trúc

Cây nhị phân là một cấu trúc dữ liệu quen thuộc, hỗ trợ các thao tác như tìm kiếm, chèn và xóa. Điểm yếu chí mạng của cây nhị phân tìm kiếm thông thường là khi dữ liệu không đủ ngẫu nhiên, hình dạng cây có thể mất cân bằng, kéo theo hiệu suất thuật toán giảm mạnh.

Skip list là một cấu trúc dữ liệu ra đời năm 1987. Với các thao tác tìm kiếm, chèn và xóa, thời gian kỳ vọng đều là $\mathcal{O}(\log n)$, gần như có thể thay thế các cây cân bằng trong nhiều tình huống. Điểm quan trọng nhất là độ phức tạp cài đặt của skip list thấp hơn nhiều so với AVL tree, red-black tree và các cấu trúc cùng loại. Vì vậy nó có lợi thế rõ rệt cả về khả năng hiểu lẫn khả năng phổ biến.

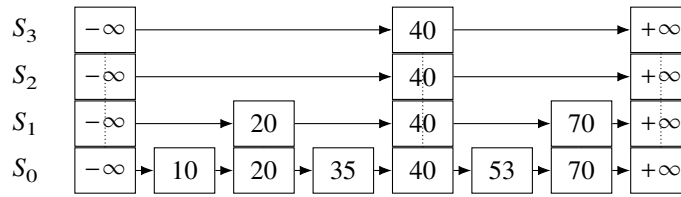
Một skip list gồm nhiều danh sách liên kết

$$S_0, S_1, S_2, \dots, S_h$$

và thỏa ba điều kiện:

- Mỗi danh sách đều chứa hai phần tử đặc biệt $-\infty$ và $+\infty$.
- S_0 chứa tất cả phần tử, và các phần tử trong mỗi danh sách được sắp xếp tăng dần.
- Với mọi $i > 0$, tập phần tử của S_i là tập con của tập phần tử trong S_{i-1} .

Hình trong bản gốc minh họa một skip list có bốn tầng S_0 tới S_3 . Các phần tử xuất hiện ở tầng cao cũng xuất hiện ở mọi tầng thấp hơn, tạo thành những “cột” thẳng đứng. Khi tìm kiếm, ta có thể đi ngang trên tầng cao để bỏ qua nhiều phần tử, rồi đi xuống dần khi cần tinh chỉnh vị trí.



Nút ở tầng cao tạo thành đường tắt, còn S_0 vẫn giữ toàn bộ thứ tự.

Figure 1: Ví dụ skip list bốn tầng: các cột cao hơn xuất hiện thưa dần theo xác suất.

2 Các thao tác cơ bản

2.1 Tìm kiếm

Mục tiêu là tìm một phần tử x trong skip list. Quy trình như sau:

1. Bắt đầu từ đầu danh sách ở tầng cao nhất S_h .
2. Giả sử vị trí hiện tại là p , nút bên phải mà p trỏ tới là q , và giá trị của q là y . So sánh y với x :
 - nếu $x = y$, tìm kiếm thành công;
 - nếu $x > y$, di chuyển sang phải tới q ;
 - nếu $x < y$, di chuyển xuống một tầng.
3. Nếu đã ở tầng đáy S_0 mà vẫn cần đi xuống, tìm kiếm thất bại.

Bản gốc dùng ví dụ tìm phần tử 53: đường đi bắt đầu từ tầng cao nhất, đi ngang qua các nút nhỏ hơn 53, rồi hạ tầng khi nút tiếp theo đã vượt mục tiêu. Đây chính là cơ chế “nhảy” làm skip list nhanh hơn danh sách liên kết đơn.

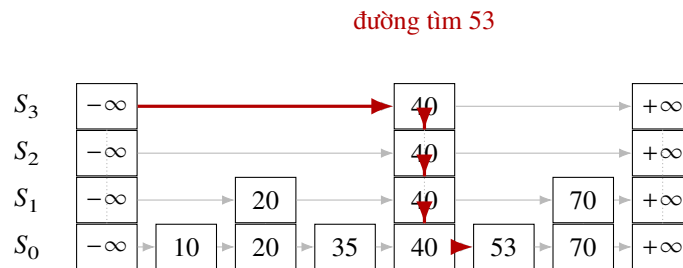


Figure 2: Tìm 53: đi ngang khi nút kế tiếp chưa vượt mục tiêu, và đi xuống khi cần tinh chỉnh.

2.2 Chèn

Chèn một phần tử x tương đương với việc thêm một cột liên tiếp bắt đầu từ tầng S_0 . Có hai tham số cần xác định: vị trí của cột và chiều cao của cột.

Vị trí được xác định bằng thao tác tìm kiếm: tìm phần tử lớn nhất $y < x$. Do mọi danh sách đều tăng dần, x phải được chèn ngay sau y .

Chiều cao của cột quan trọng hơn và khó xác định hơn. Nếu chọn không tốt, hiệu suất của cấu trúc có thể thay đổi lớn. Để sau khi chèn dữ liệu, các thao tác vẫn giữ thời gian kỳ vọng $\mathcal{O}(\log n)$, skip list

dùng một quyết định ngẫu nhiên hóa. Với một hằng số $p \in (0, 1)$, mô-đun quyết định có dạng:

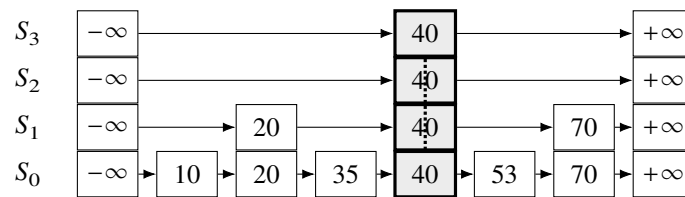
$$r \leftarrow \text{random}(0, 1), \quad \begin{cases} \text{thực hiện phương án A,} & r < p, \\ \text{thực hiện phương án B,} & r \geq p. \end{cases}$$

Ban đầu chiều cao cột là 1. Mỗi lần mô-đun trả về A, tăng chiều cao thêm 1 và tiếp tục gieo. Khi mô-đun trả về B ở lần thứ i , dừng lại và chèn một cột cao i . Khi đó:

$$\Pr(\text{chiều cao cột} \geq k) = p^{k-1}.$$

Nếu chiều cao mới lớn hơn chiều cao hiện tại h của skip list, cần thêm các danh sách mới phía trên để cấu trúc vẫn thỏa các điều kiện đã nêu.

Bản gốc minh họa thao tác chèn 40: trước hết tìm phần tử lớn nhất nhỏ hơn 40, rồi chèn một cột cao 4. Nếu chiều cao này vượt chiều cao cũ, một tầng mới được tạo thêm.



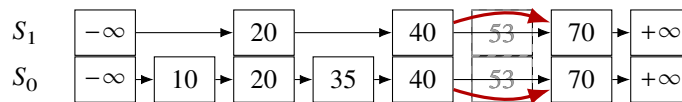
Chiều cao ngẫu nhiên bằng 4, nên nút 40 được nối vào bốn tầng.

Figure 3: Chèn 40: sau khi tìm vị trí, tạo một cột có chiều cao do mô-đun ngẫu nhiên quyết định.

2.3 Xóa

Xóa phần tử x gồm ba bước:

1. Tìm vị trí của x trong skip list; nếu không tồn tại thì kết thúc.
2. Xóa toàn bộ cột chứa x khỏi các danh sách.
3. Xóa các tầng trên cùng đã trở thành rỗng, tức chỉ còn hai nút biên $-\infty$ và $+\infty$.



Xóa toàn bộ cột 53, rồi nối lại các tiền nhiệm với hậu nhiệm trên từng tầng.

Figure 4: Xóa một phần tử khỏi skip list là xóa cả cột của nó và vá lại các liên kết ngang.

2.4 Tìm kiếm có nhớ

Tìm kiếm có nhớ, thường gọi là *search with fingers*, là cách tận dụng thông tin từ lần tìm kiếm trước để hỗ trợ lần tìm kiếm kế tiếp. Nếu hai phần tử được tìm liên tiếp nằm gần nhau trong thứ tự của skip list, kỹ thuật này có thể giảm độ phức tạp kỳ vọng xuống $\mathcal{O}(\log k)$, trong đó k là khoảng cách thứ tự giữa hai phần tử ấy.

Giả sử lần trước tìm phần tử i , lần này cần tìm phần tử j . Ta dùng mảng *update* để ghi lại, trên mỗi tầng, vị trí bên phải nhất mà đường tìm kiếm đã nhảy tới khi tìm i . Khi tìm j :

- Nếu $i \leq j$, bắt đầu từ tầng S_0 , duyệt ngược lên trong mảng *update* cho tới khi gặp một vị trí có nút bên phải mang giá trị ít nhất j , rồi từ đó chạy quy trình tìm kiếm thông thường.
- Nếu $i > j$, cũng duyệt từ tầng S_0 lên trên, tìm vị trí thích hợp mà từ đó phần tử bên trái hoặc hiện tại không vượt quá j , rồi tiếp tục tìm kiếm thông thường.

Kỹ thuật này đặc biệt hiệu quả với dữ liệu có tương quan mạnh giữa các lần truy vấn liên tiếp. Với dữ liệu gần như ngẫu nhiên, chi phí duy trì và quét thông tin nhớ có thể không đem lại lợi ích.

3 Phân tích độ phức tạp

Các đại lượng quan trọng của skip list đều được hiểu theo kỳ vọng:

Bộ nhớ	$\mathcal{O}(n)$,
Chiều cao	$\mathcal{O}(\log n)$,
Tìm kiếm	$\mathcal{O}(\log n)$,
Chèn	$\mathcal{O}(\log n)$,
Xóa	$\mathcal{O}(\log n)$.

Lý do cần nói “kỳ vọng” là vì phân tích của skip list dựa trên xác suất. Có thể xuất hiện trường hợp xấu, nhưng xác suất ấy rất nhỏ.

3.1 Bộ nhớ

Giả sử có n phần tử. Theo công thức chiều cao, xác suất một phần tử xuất hiện ở tầng S_i là p^{i-1} . Vì vậy số phần tử kỳ vọng trên tầng i là np^{i-1} , và tổng số nút kỳ vọng là

$$\sum_{i \geq 1} np^{i-1} = \frac{n}{1-p}.$$

Khi $p < 1/2$, tổng này nhỏ hơn $2n$. Do đó bộ nhớ kỳ vọng là $\mathcal{O}(n)$.

3.2 Chiều cao

Tương tự, ở tầng cao $1 + 3 \log_{1/p} n$, số phần tử kỳ vọng xấp xỉ

$$np^{3 \log_{1/p} n} = \frac{1}{n^2}.$$

Khi n lớn, giá trị này gần 0, nên xác suất skip list còn phần tử ở cao hơn mức ấy là rất nhỏ. Vì vậy chiều cao kỳ vọng thuộc cấp $\mathcal{O}(\log n)$.

3.3 Tìm kiếm

Ta dùng phân tích ngược. Giả sử đang đứng ở nút đích và muốn đi ngược về nút bắt đầu ở góc trái trên. Độ dài đường đi ngược này tương ứng với số bước của tìm kiếm.

Xét một nút ở tầng i . Nếu cột của nó đúng cao i , sự kiện này có xác suất $1 - p$, và đường đi ngược phải đến từ bên trái. Nếu cột cao hơn i , sự kiện này có xác suất p , và đường đi ngược đến từ phía trên.

Gọi $C(k)$ là số bước kỳ vọng để đi ngược lên k tầng, kể cả các bước ngang. Khi đó:

$$C(0) = 0,$$

$$C(k) = (1 - p)(1 + C(k)) + p(1 + C(k - 1)).$$

Suy ra

$$C(k) = \frac{1}{p} + C(k - 1) = \frac{k}{p}.$$

Vì chiều cao k ở cấp $\mathcal{O}(\log n)$, thời gian tìm kiếm kỳ vọng cũng là $\mathcal{O}(\log n)$. Với tìm kiếm có nhớ, lập luận tương tự cho kết quả $\mathcal{O}(\log k)$, trong đó k là khoảng cách giữa hai phần tử được tìm liên tiếp.

3.4 Chèn và xóa

Chèn và xóa gồm hai phần: tìm vị trí và cập nhật liên kết. Phần tìm kiếm có độ phức tạp $\mathcal{O}(\log n)$, còn phần cập nhật tỷ lệ với chiều cao của skip list, cũng là $\mathcal{O}(\log n)$. Do đó cả chèn lẫn xóa đều có thời gian kỳ vọng $\mathcal{O}(\log n)$.

3.5 Kết quả thử nghiệm

Bản gốc chạy 10^6 thao tác ngẫu nhiên để so sánh các giá trị p :

p	thời gian	cao TB	số nút	nhảy tìm	nhảy chèn/xóa
2/3	0.0024690 ms	3.0049	123339	39.878	41.604/41.566
1/2	0.0020180 ms	1.9956	68327	27.807	29.947/29.072
1/e	0.0019870 ms	1.5844	57027	27.332	28.238/28.452
1/4	0.0021720 ms	1.3304	47828	28.726	29.472/29.664
1/8	0.0026880 ms	1.1443	44203	35.147	35.821/36.007

Kết quả cho thấy $p = 1/2$ và $p = 1/e$ có hiệu suất thời gian cao. Nếu bộ nhớ là ràng buộc chặt, có thể chọn p nhỏ hơn, chẳng hạn $1/4$.

Với tìm kiếm có nhớ, bản gốc ghi nhận:

p	dữ liệu	không nhớ	có nhớ	nhảy không nhớ	nhảy có nhớ
0.5	ngẫu nhiên	0.0020150	0.0020790	23.262	26.509
0.5	có tương quan	0.0008440	0.0006880	26.157	4.932

Đơn vị thời gian là mili giây trên mỗi thao tác. Khi các khóa được tìm liên tiếp gần nhau, tìm kiếm có nhớ đem lại lợi ích rõ rệt. Khi dữ liệu ngẫu nhiên mạnh, kỹ thuật này không những không nhanh hơn mà còn có thể trở thành nút thắt do số lần nhảy tăng.

4 Ứng dụng của skip list

Hiệu suất cao và độ phức tạp cài đặt thấp làm skip list hữu ích trong nhiều bài toán thực tế, đặc biệt khi thời gian lập trình rất hạn chế. Bài viết dùng hai bài thi để minh họa.

4.1 NOI 2004 Day 1: Cashier

Bài Cashier yêu cầu duy trì hồ sơ lương nhân viên dưới bốn thao tác: thêm nhân viên mới, tăng toàn bộ lương, giảm toàn bộ lương và hỏi mức lương lớn thứ k . Khi lương bị giảm xuống dưới ngưỡng hợp đồng, nhân viên rời công ty và phải bị xóa khỏi cấu trúc.

Gọi R là miền giá trị lương và N là số nhân viên. Ba hướng giải được so sánh như sau:

Cấu trúc	I	A	S	F
segment tree theo lương	$\mathcal{O}(\log R)$	$\mathcal{O}(1)$	$\mathcal{O}(\log R)$	$\mathcal{O}(\log R)$
splay tree theo lương	$\mathcal{O}(\log N)$	$\mathcal{O}(1)$	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$
skip list	$\mathcal{O}(\log N)$	$\mathcal{O}(1)$	$\mathcal{O}(\log N)$	$\mathcal{O}(\log N)$

Kết quả chạy thực tế trong bản gốc, đơn vị giây:

Cấu trúc	$T1$	$T2$	$T3$	$T4$	$T5$	$T6$	$T7$	$T8$	$T9$	$T10$
segment tree	0.000	0.000	0.000	0.031	0.062	0.094	0.109	0.203	0.265	0.250
splay tree	0.000	0.000	0.016	0.062	0.047	0.125	0.141	0.360	0.453	0.422
skip list	0.000	0.000	0.000	0.047	0.062	0.109	0.156	0.368	0.438	0.375

Segment tree tỏ ra nhanh nhất trong dữ liệu này, nhưng nó dựa trực tiếp vào miền khóa. Ở bài gốc, R chỉ khoảng 10^5 , nên dùng segment tree còn hợp lý. Nếu miền khóa lớn hơn nhiều, hoặc khóa không phải số nguyên, segment tree trở nên khó áp dụng. Khi đó các cấu trúc dựa trên phần tử như splay tree và skip list thích hợp hơn. Kết quả thực nghiệm cũng cho thấy skip list không kém splay tree, trong khi cài đặt đơn giản hơn.

4.2 HNOI 2004 Day 1: Pet Adoption

Bài Pet Adoption duy trì một tập thú nuôi hoặc một tập người nhận nuôi. Mỗi lần có đối tượng mới đến, nếu phía còn lại không rỗng thì ghép với đối tượng có trị đặc trưng gần nhất; nếu hòa, chọn trị nhỏ hơn. Miền trị đặc trưng lên tới 2^{31} .

Điểm khác lớn nhất so với Cashier là miền khóa quá rộng. Segment tree gặp áp lực bộ nhớ lớn; dù dùng cách cấp phát nút động, nếu các khóa phân tán thì độ phức tạp bộ nhớ có thể tiến gần $\mathcal{O}(N \log N)$. Nếu mở rộng bài toán để khóa là số thực, segment tree gần như mất tác dụng.

Skip list phù hợp tự nhiên với bài này. Hầu như không cần sửa thuật toán chuẩn, có thể viết nhanh nếu đã quen, hiệu suất tốt, và quan trọng hơn là nó không phụ thuộc chặt vào kiểu khóa. Tính mở rộng này là một ưu điểm lớn của skip list trong thi đấu.

5 Kết luận

Skip list là một cấu trúc dữ liệu mới mẻ so với nhiều cây cân bằng kinh điển, nhưng có hiệu suất cao và độ phức tạp cài đặt thấp. So với splay tree, vốn cũng nổi tiếng vì tương đối dễ viết, skip list không hề kém về hiệu suất, và trong một số thử nghiệm còn tốt hơn.

Khi suy nghĩ về một lớp bài toán, ta thường vô thức bị giới hạn trong một phạm vi quen thuộc. Với các bài toán liên quan tới cây cân bằng, nhiều cấu trúc mạnh như red-black tree hay AVL tree đã được tạo ra, nhưng chúng vẫn nằm trong khuôn khổ “cây” và thường có cài đặt phức tạp. Skip list cho thấy một cấu trúc không liên quan trực tiếp đến cây vẫn có thể thực hiện chức năng tương tự.

Cái tên “skip list” vì thế không chỉ mô tả hình dạng dữ liệu, mà còn gợi ra một cách nghĩ: khi gặp bế tắc, hãy quay về bản chất của vấn đề và thử nhảy ra khỏi khuôn mẫu quen thuộc.

6 Phụ lục

6.1 Vì sao p bằng một trên e thường hiệu quả?

Từ phân tích độ phức tạp, thời gian phụ thuộc vào số lần nhảy, xấp xỉ

$$\frac{k}{p},$$

trong đó k là chiều cao skip list. Mà

$$k = \Theta(\log_{1/p} n).$$

Bỏ qua hằng số chỉ phụ thuộc vào n , cần tối ưu

$$f(p) = \frac{1}{p} \log_{1/p} n.$$

Đặt $x = 1/p$. Khi đó, về phần phụ thuộc vào x ,

$$g(x) = \frac{x}{\ln x}.$$

Đạo hàm cho

$$g'(x) = \frac{\ln x - 1}{(\ln x)^2}.$$

Do đó $g'(x) = 0$ khi $x = e$, tức $p = 1/e$. Đây là lý do thực nghiệm thường cho thấy $p = 1/e$ đạt hiệu suất thời gian rất tốt.

6.2 Ghi chú về phụ lục chương trình và hình vẽ

Bản gốc có bảng so sánh lấy từ bài “Skip Lists: A Probabilistic Alternative to Balanced Trees” của William Pugh, một chương trình Pascal cài đặt skip list, cùng các hình minh họa thao tác tìm kiếm, chèn và xóa. Các hình thao tác chính đã được vẽ lại ở các phần trên. Mã Pascal không phải nội dung cần dịch, nên không được chép lại đầy đủ.

6.3 Hai đề bài được dùng làm ví dụ

Cashier yêu cầu xử lý tối đa 100000 lệnh thêm nhân viên và 100000 truy vấn hạng lương. Có bốn loại lệnh: thêm hồ sơ với lương ban đầu, tăng lương toàn công ty, giảm lương toàn công ty và hỏi lương lớn thứ k . Cần in đáp án cho từng truy vấn và tổng số nhân viên đã rời công ty.

Pet Adoption yêu cầu xử lý tối đa 80000 sự kiện. Mỗi sự kiện là một thú nuôi hoặc một người nhận nuôi với trị đặc trưng dương nhỏ hơn 2^{31} . Nếu phía đối ứng đang chờ, ghép với trị gần nhất và cộng độ bất mãn $|a - b|$; nếu không, đưa đối tượng mới vào tập chờ. Cần in tổng độ bất mãn lấy modulo 1000000.

References

- [1] William Pugh. *Skip Lists: A Probabilistic Alternative to Balanced Trees*.
- [2] William Pugh. *A Skip List Cookbook*.